

Vector Search for pgrust: Algorithm & Benchmark Survey

Oct 4, 2026 · @Jody Schnarr

Summary

Build pgvector-compatible types and HNSW first, then add a clustering index (IVF + RaBitQ) as the performance flagship, and make planner-integrated filtering the feature no extension can match.

- **Compatibility is the floor.** pgvector's `vector`, `halfvec`, `bit` and `sparsevec` types plus its HNSW and IVFFlat page formats let existing databases boot on pgrust unchanged. New index methods sit alongside, the way pgvector's `scale`'s `diskann` and AlloyDB's ScaNN do.
- **The strongest Postgres-native results today come from clustering + quantization, not graphs.** VectorChord (IVF + RaBitQ) and AlloyDB ScaNN both report large wins over pgvector HNSW on build time, memory and often QPS, mainly because sequential page scans suit a page-based engine better than random graph hops.
- **Inside a database, system overhead dominates.** A 2026 SIGMOD study found page access and TID indirection, not distance maths, consume most query time, so library benchmark rankings do not transfer directly.
- **pgrust's edge is the engine itself:** planner control over pre/inline/post filtering, threads for intra-query parallelism, and pgrcolumnar for SIMD-friendly vector storage.
- **Licences:** pgvector and pgvector's `scale` are permissively licensed; VectorChord is AGPLv3 or ELv2, and its AGPLv3 option is compatible with pgrust's own AGPLv3, so its code can be reused under AGPL terms. RaBitQ itself is a published algorithm anyone can implement.

Compatibility baseline: pgvector

The target is pgvector's 0.8.x line, which secondary sources put at 0.8.6 as of July 2026; confirm against the [pgvector repo](#) before freezing formats.

Area	What pgrust must match	Notes
Types	<code>vector</code> , <code>halfvec</code> , <code>bit</code> , <code>sparsevec</code>	halfvec indexes up to 4,000 dims; binary quantization up to 64,000 (pgxn)
Operators	<code><-></code> L2, <code><#></code> negative inner product, <code><=></code> cosine, <code><+></code> L1, <code><~></code> Hamming, <code><%></code> Jaccard	<code><#></code> is negated because Postgres index scans only order ascending
Index AMs	<code>hnsw</code> , <code>ivfflat</code> and their opclasses	Page layout and WAL records must be byte-compatible for in-place boot
Iterative scans	<code>hnsw.iterative_scan</code> , <code>max_scan_tuples</code> , <code>ivfflat.max_probes</code>	Added in 0.8.0: keeps scanning until enough rows pass the filter (README)
Quantization	<code>binary_quantize()</code> expression indexes + re-ranking	No native SQ/PQ in pgvector itself

Two known pgvector limits are worth not inheriting in new index methods. Its HNSW requires a node's neighbour list to fit on one 8 KB page, which forces a trade-off between graph degree and layer depth. And a 2026 study found its halfvec and binary quantization shrink indexes by up to roughly 12x but give no consistent QPS gain, because HNSW in Postgres stays bound by random page accesses ([Lu et al.](#)).

Index algorithms compared

Inside a page-based engine, clustering indexes (IVF, ScaNN) usually beat graphs on build time, memory and update cost; graphs keep an edge at high dimensions and for selective filters with large k .

Family	Examples	Access pattern in Postgres	Build / updates	Memory	Best fit
Graph (in-memory)	HNSW (pgvector, hnswlib)	Random page hop per neighbour; TID indirection to heap	Slow build; pgvector reportedly needs 50+ hours for 100M vectors on 16 vCPUs (VectorChord)	Highest	Index fits in RAM, high-dim, low-latency
Graph (disk-resident)	DiskANN / Vamana (pgvectorscale)	Fewer, larger reads; compressed vectors guide traversal	Moderate	Low with SBQ	Larger-than-RAM datasets, tens of millions+
Clustering (IVF)	IVFFlat, VectorChord vchordrq	Sequential scan of posting lists; SIMD-friendly	Fast; k-means can run externally on GPU	Low with quantization	Large datasets, heavy writes
Tree-quantization	ScaNN (AlloyDB)	Multi-level clustering, sequential leaf pages	Google cites up to 16x faster creation than HNSW	Up to 4x smaller than HNSW	Low-dim or massive datasets; filtered search

The in-database evidence is consistent. Google's SIGMOD 2026 study measured ScaNN 2-3x faster than HNSW variants on low-dimensional data, with the gap narrowing on 1536-dim and 768-dim embeddings ([Lu et al.](#)). The same study measured HNSW builds at 20-73 minutes versus 2-18 minutes for ScaNN on 5-10M vectors, with indexes 5-20x larger.

Quantization

RaBitQ is the strongest candidate: it gives unbiased distance estimates with a proven error bound, so the engine can decide which candidates to re-rank without hand-tuned parameters.

Method	Bits/dim	Key property	Where used
RaBitQ (SIGMOD 2024)	1	Random rotation, then sign bits; unbiased estimator with error bound	VectorChord

Method	Bits/dim	Key property	Where used
Extended RaBitQ (SIGMOD 2025)	B (any)	Asymptotically optimal error bound; reported 1.3-3.1x lower error than LVQ	Research; successors such as SAQ
Statistical Binary Quantization	1-2	Improves accuracy over plain binary quantization	pgvectorscale <code>diskann</code>
Binary quantization + re-rank	1	Simple; needs deep re-rank lists	pgvector via <code>binary_quantize()</code>
Scalar (int8 / fp16)	8-16	Low recall loss, modest compression	pgvector <code>halfvec</code> ; ScaNN SQ8 default
PQ / OPQ	~0.5-1	Codebook-based; no error bound	FAISS; pgvectorscale deprecated its PQ

Quantization pays off far more in clustering indexes than in graphs. In the Google study, ScaNN with SQ8 plus PCA ran 5-50x faster than unquantized ScaNN, while quantizing pgvector HNSW gave no reliable QPS gain. A pgvector issue reports RaBitQ shrinking an IVFFlat index from 7,820 MB to 281 MB on 1M 1536-dim vectors ([#1003](#)).

Binary quantization alone has a failure mode: AWS reported Cohere 10M needing 3,000 re-rank candidates per query, dropping throughput to 16 QPS ([AWS](#)). Error-bounded methods like RaBitQ are the fix.

Filtered search

No single filtered-search strategy wins: the best choice depends on filter selectivity, how the filter correlates with vector proximity, and k, so it belongs in the planner, not hard-coded in the index.

Strategy	How it works	Wins when	Source
Pre-filter + exact scan	Filter first, brute-force the survivors	Extremely selective filters (below ~0.01% in one study)	Lu et al.
Iterative scan / Sweeping	Traverse the normal graph, filter results, resume if short	Broad filters (high selectivity)	pgvector 0.8.0

Strategy	How it works	Wins when	Source
ACORN (filter-first)	Traverse only predicate-passing nodes, with 2-hop expansion	Selective filters; 2-1,000x higher QPS in library tests	ACORN, SIGMOD 2024
NaviX-Directed	ACORN-1 plus ranked 2-hop expansion, chosen adaptively per step	Best graph method at 5-50% selectivity in Postgres	Lu et al.
Filtered ScaNN / IVF	Filter only inside chosen leaves; SIMD-score survivors	Low-dim data; negative correlation (filter excludes nearest vectors)	AlloyDB
Specialised hybrid indexes	Tag-aware inverted + spatial index	Simple tag filters; won NeurIPS'23 filtered track (ParlayANN IVF ² , 11x the FAISS baseline)	Big-ANN '23

Key findings from the Postgres study that should shape pgrust's design:

- In Postgres, system overhead (buffer pins, locks, copying vectors, heap TID lookups) took 56-84% of CPU cycles even single-threaded. Library rankings do not transfer.
- Resolving index TIDs to heap TIDs dominated filter-first cost (60-75% of cycles) until the authors added an in-memory translation map.
- Under negative correlation, graph methods lost 44-89% of throughput at 1% selectivity while ScaNN gained slightly.
- The authors propose columnar engines to host both the index and the filter columns. pgrust already has pgrcolumnar.

Postgres-native projects

All three serious successors to pgvector HNSW keep pgvector's data types and add a new index method, which validates the compatibility-plus-new-AM plan.

Project	Approach	Reported results (vendor)	Language	Licence
VectorChord	IVF + RaBitQ, re-ranking, external k-means; experimental DiskANN + RaBitQ	100M vectors indexed in under 20 min on 16 vCPUs; claims up to 5x faster queries than pgvector HNSW; 3B+ vectors in production	Rust (pgrx)	AGPLv3 or ELv2
pgvectorscale	StreamingDiskANN + Statistical Binary Quantization; label-based filtering	28x lower p95 latency than Pinecone at 99% recall on 50M Cohere vectors	Rust (pgrx)	PostgreSQL
ScaNN for AlloyDB	Multi-level tree quantization; adaptive and inline filtering in the planner	Up to 6x faster queries, 10x faster filtered queries, 16x faster builds, 4x less memory than HNSW	C++	Proprietary (the ScaNN library is Apache-2.0)
pgvector	HNSW, IVFFlat, iterative scans	Baseline	C	PostgreSQL

Vendor numbers use favourable setups, so treat them as directional. Independent testing finds DiskANN only beats HNSW once the index no longer fits in RAM ([The Build](#)). Because pgrust is AGPLv3, VectorChord's AGPLv3 option allows its code to be reused in pgrust under the same terms; anything that should stay permissively licensed should implement RaBitQ from the papers instead.

Benchmarks

Use VectorDBBench for system-level comparisons inside Postgres and big-ann-benchmarks for algorithm-level comparisons; never rank on unfiltered QPS on a static dataset alone.

Benchmark	Measures	Postgres coverage	Current leaders / notes
VectorDBBench	Recall vs QPS, p99 latency, load time, filtered and streaming cases, cost	Built-in clients for pgvector (HNSW, IVFFlat), pgvectorscale, pgdiskann, AlloyDB and VectorChord	Closest to real workloads; run it on your own hardware
big-ann-benchmarks NeurIPS'23	QPS at 90% recall@10 on filtered, out-of-distribution, sparse and streaming tracks	None (libraries)	Ongoing filter-track leaderboard: Pinecone 85,491 QPS, Zilliz 84,596, ParlayANN IVF ² 37,902, against a FAISS baseline of 3,200
ann-benchmarks	Recall vs QPS, in-memory, unfiltered	pgvector entry	Useful for kernel-level tuning only

What to measure for pgrust:

1. Recall@10 and @100 against QPS at fixed recall targets (95%, 99%).
2. Index build time and memory, at 1M, 10M and 100M vectors.
3. Filtered queries across selectivities from 0.1% to 90%, with positive and negative correlation.
4. Throughput while inserts and deletes run concurrently.
5. Real embeddings only: one test found synthetic random vectors gave 3.8% HNSW recall against 91.8% on real data ([dev.to](#)).

pgrust advantages and roadmap

pgrust can fix in the engine what extensions can only work around: the costs the Postgres study found dominant (page pins, TID indirection, single-threaded scans, filter-blind planning) are all internal to the server.

Advantages a native module has over any extension:

- **Planner ownership.** Choose pre-filter, inline filter or iterative scan per query from selectivity and correlation estimates, as AlloyDB does with adaptive filtering.
- **Threads.** pgrust is moving to multithreaded internals, so it can offer parallel builds and intra-query parallel search, which pgvector cannot.
- **Storage layout.** Store quantized codes and filter columns in pgrcolumnar for sequential SIMD scans, which the study's authors name as the promising direction.
- **No ABI constraints.** Native heap-TID translation maps, page sizes and neighbour lists that do not have to fit one 8 KB page.

Recommended phases:

1. **Compatibility.** Ship `vector`, `halfvec`, `bit`, `sparsevec`, all operators, and byte-compatible `hnsw` and `ivfflat` with iterative scans. Gate: pgvector's own regression suite passes, and a pgvector 0.8.x data directory boots and queries correctly.
2. **Flagship index.** Add an IVF + RaBitQ access method with error-bound re-ranking and parallel or external k-means. Gate: beats pgvector HNSW on build time and memory at 10M+ vectors with equal recall in VectorDBBench.
3. **Planner-integrated filtering.** Selectivity- and correlation-aware choice among pre-filter, inline filter (NaviX-Directed style for graphs, leaf-level for IVF) and iterative scan. Gate: no strategy loses more than 2x to the best fixed strategy across a 0.1-90% selectivity sweep.
4. **Columnar and parallel execution.** Quantized codes in pgrcolumnar, intra-query parallel probes, batch query APIs. Gate: measured reduction in system-overhead share of cycles.
5. **Disk-resident option (only if needed).** DiskANN-style graph for datasets far beyond RAM, informed by pgvectorscale's permissively licensed design.

Open questions: whether upstream pgrust plans its own vector work or C-ABI extension loading; and whether the workload is dominated by high-dimensional text embeddings (favouring a graph option) or larger, lower-dimensional data (favouring IVF).

Sources

- [pgvector README](#) and [pgxn 0.8.0 docs](#)
- [Lu et al., An In-Depth Study of Filter-Agnostic Vector Search on a PostgreSQL Database System, SIGMOD 2026](#)
- [Patel et al., ACORN, SIGMOD 2024](#)
- [Gao & Long, RaBitQ, SIGMOD 2024](#) and [Extended RaBitQ, SIGMOD 2025](#)
- [VectorChord 1.0 announcement](#) and [VectorChord licence](#)
- [pgvectorscale repo](#) and [licence](#)
- [AlloyDB: choose a vector index](#) and [ScaNN vs pgvector HNSW](#)
- [pgvector issue #1003: RaBitQ support](#)
- [AWS: binary quantization on Aurora pgvector](#)
- [Big-ANN NeurIPS'23 leaderboard](#) and [results paper](#)
- [VectorDBBench](#)
- [The Build: pgvectorscale benchmark](#)
- [pgrust repo](#)